

Actividad 2: Instalación completa de WordPress con Chef y pruebas unitarias y de integración

Integrantes del Grupo 6:

- Hector Noel Medina Giron
- María Dolores Aguera Soriano
- Moisés Alvin Miguel Flores
- Herson Mena Mora

Tabla de contenidos

Introducción.....	2
Objetivos.....	2
Estructura del proyecto	3
VagrantFile	4
Chef/Cookbooks.....	4
Principal	4
Mi_apache	5
Mi_mysql	5
Mi_php	5
Mi_wordpress.....	5
Ejecución de proyecto	7
Pruebas automatizadas.....	7
Pruebas unitarias	7
Pruebas de integración	8

Introducción

El presente informe documenta el proceso de automatización de la instalación y configuración de un servidor WordPress utilizando Chef como herramienta de gestión de configuración y Vagrant como entorno de virtualización. El proyecto forma parte de la asignatura Herramientas de Automatización de Despliegues de TechOps Solutions, y su objetivo es demostrar el uso de Chef para orquestrar despliegues reproducibles de infraestructura LAMP.

WordPress es un sistema de gestión de contenidos (CMS) escrito en PHP que requiere un entorno LAMP (Linux, Apache, MySQL y PHP) para su funcionamiento. Automatizar su instalación garantiza que cualquier miembro del equipo pueda reproducir el entorno de forma idéntica, eliminando la variabilidad manual y los errores asociados a configuraciones ad hoc.

Objetivos

Los objetivos concretos de la actividad son los siguientes:

- Automatizar mediante Chef la instalación y configuración del stack LAMP completo (Apache, MySQL y PHP) en una máquina virtual Ubuntu 22.04 gestionada con Vagrant.
- Descargar, descomprimir y configurar WordPress de forma totalmente desatendida, incluyendo la instalación inicial mediante WP-CLI para que el servidor muestre el primer blog directamente al acceder desde un navegador.
- Estructurar el proyecto en cookbooks modulares e independientes para cada componente del stack.
- Implementar al menos cinco casos de pruebas unitarias con ChefSpec y cinco casos de pruebas de integración con InSpec mediante Test Kitchen, que validen el correcto funcionamiento del aprovisionamiento en sistemas Ubuntu.

Estructura del proyecto

El proyecto se organiza en un directorio raíz que contiene el Vagrantfile, el fichero de políticas de Chef (Policyfile.rb), la configuración de Test Kitchen (kitchen.yml) y la carpeta chef/cookbooks con los cookbooks desarrollados. Las pruebas unitarias residen en spec/unit/recipes, y las pruebas de integración en test/integration/wordpress/.

```
unir-devops-chef-wordpress/
├─ Vagrantfile
├─ Policyfile.rb
├─ kitchen.yml
├─ chef/
│   └─ cookbooks/
│       ├── principal/      <- Orquestador
│       ├── mi_apache/     <- Instalación Apache
│       ├── mi_mysql/      <- Instalación MySQL
│       ├── mi_php/        <- Instalación PHP
│       └─ mi_wordpress/   <- Descarga y configuración WordPress
├─ spec/
│   └─ unit/
│       └─ recipes
└─ test/
    └─ integration/
        └─ wordpress/
            └─ wordpress_spec.rb
```

La tabla 1 resume el propósito de cada cookbook implementado.

Tabla 1. Descripción de los cookbooks del proyecto.

Cookbook	Versión	Descripción
principal	0.1.0	Orquestador: incluye los cuatro cookbooks en el orden correcto
mi_apache	0.1.0	Instala Apache 2, configura el virtual host y activa mod_rewrite
mi_mysql	0.1.0	Instala MySQL Server, crea la base de datos y el usuario de WordPress
mi_php	0.1.0	Instala PHP con las extensiones requeridas por WordPress

Nota: Todos los cookbooks son de desarrollo propio y requieren Chef >= 14.0.

VagrantFile

El fichero Vagrantfile define una máquina virtual denominada server-wordpress basada en la box bento/ubuntu-22.04, con 1024 MB de RAM, IP privada 192.168.33.20 y redirección del puerto 80 del guest al puerto 80 del host. El aprovisionamiento se realiza en dos fases: primero un provisioner shell instala el cliente Chef Infra, y luego un provisioner chef_solo aplica la receta principal con los atributos de WordPress.

```
serverwordpress.vm.provision 'chef_solo' do |chef|
  chef.cookbooks_path = 'chef/cookbooks'
  chef.add_recipe      'principal'
  chef.arguments       = '--chef-license accept -l info'
  chef.json = {
    'wordpress' => {
      'url'          => 'http://192.168.33.20',
      'document_root' => '/var/www/wordpress',
      'db_name'       => 'wordpress_db',
      'db_user'        => 'wp_user',
      'db_password'    => 'wp_password',
      'title'          => 'TechOps Solutions, S.A',
      'admin_user'     => 'admin',
      'admin_password' => 'admin123',
      'admin_email'    => 'admin@example.com'
    }
  }
end
```

Chef/Cookbooks

Principal

El cookbook principal actúa como punto de entrada del aprovisionamiento. Su receta default.rb incluye los cuatro cookbooks específicos en el orden de dependencia correcto: Apache primero, después MySQL, luego PHP y finalmente WordPress. Este orden es

necesario porque el módulo PHP de Apache (libapache2-mod-php) notifica el reinicio del servicio apache2, que debe existir previamente.

Mi_apache

Este cookbook actualiza la caché apt, instala el paquete apache2, crea el directorio raíz de documentos con permisos para www-data, habilita y arranca el servicio, despliega la configuración del virtual host desde una plantilla ERB, deshabilita el sitio por defecto (000-default.conf), activa el sitio wordpress.conf y habilita el módulo mod_rewrite. La plantilla vhost.conf.erb configura el DocumentRoot y el bloque AllowOverride All, necesario para los enlaces permanentes de WordPress.

Mi_mysql

El cookbook mi_mysql instala mysql-server, habilita el servicio y ejecuta tres comandos SQL con guarda not_if para garantizar la idempotencia: creación de la base de datos wordpress_db, creación del usuario wp_user con su contraseña y concesión de privilegios completos sobre la base de datos. El uso de not_if permite que el aprovisionamiento se ejecute varias veces sin producir errores ni duplicados.

Mi_php

Se instalan ocho paquetes PHP necesarios para WordPress: php, php-mysql, php-curl, php-gd, php-mbstring, php-xml, php-zip y libapache2-mod-php. Este último paquete notifica el reinicio diferido (delayed) de apache2, lo que garantiza que el reinicio se produce una sola vez al final de la convergencia de Chef.

Mi_wordpress

Este cookbook encadena seis acciones: descarga del archivo comprimido de WordPress desde wordpress.org con wget, extracción al directorio /var/www/wordpress usando --strip-components=1, corrección de permisos asignando www-data como propietario,

generación del fichero wp-config.php desde una plantilla ERB con las credenciales de base de datos y las claves de seguridad (salts) generadas dinámicamente, descarga de WP-CLI, e instalación de WordPress mediante wp core install con los datos del administrador y el título del sitio. La guarda not_if sobre la instalación comprueba si WordPress ya está instalado para garantizar la idempotencia.

Gracias a WP-CLI, al finalizar el aprovisionamiento el servidor WordPress está completamente operativo y muestra el primer blog directamente, sin necesidad de completar ningún asistente manual.

```
11
12 execute 'download-wordpress' do
13   command 'wget https://wordpress.org/latest.tar.gz -P /tmp/'
14   creates '/tmp/latest.tar.gz'
15   action :run
16 end
17
18 execute 'extract-wordpress' do
19   command "tar -xzf /tmp/latest.tar.gz -C #{document_root} --strip-components=1"
20   creates "#{document_root}/wp-login.php"
21   action :run
22 end
23
24 execute 'fix-wordpress-permissions' do
25   command "chown -R www-data:www-data #{document_root}"
26   action :run
27 end
28
29 template "#{document_root}/wp-config.php" do
30   source 'wp-config.php.erb'
31   owner 'www-data'
32   group 'www-data'
33   mode '0640'
34   variables(
35     db_name: db_name,
36     db_user: db_user,
37     db_password: db_password,
38     db_host: db_host,
39     wp_url: wp_url
40   )
41   action :create
42 end
43
44 remote_file '/usr/local/bin/wp' do
45   source 'https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar'
46   mode '0755'
47   action :create_if_missing
48 end
```

Ejecución de proyecto

Correr el comando -> vagrant up

Al finalizar se mostrará la ip donde queda accesible Wordpress:

[http:// 192.168.33.20/wordpress](http://192.168.33.20/wordpress)

Pruebas automatizadas

Pruebas unitarias

Las pruebas unitarias del proyecto se ejecutan con RSpec y ChefSpec sobre Ruby.

Para poder lanzarlas es necesario tener Ruby instalado y Bundler. En la raíz del proyecto existe un Gemfile que declara las gemas de prueba chefspec y chef; tras instalar dependencias con bundle install, el comando bundle exec rspec ejecuta toda la batería usando las versiones indicadas en Gemfile.lock y evita conflictos con gemas globales del sistema.

La configuración común está en spec/spec_helper.rb: allí se carga ChefSpec y se define cookbook_path apuntando a la carpeta chef/cookbooks para que los cookbooks del repositorio se resuelvan al converger. El fichero .rspec en la raíz activa la salida en color y el formato documentation de RSpec.

Los casos de prueba viven en spec/unit/recipes/, un archivo por receta principalmente probada. Cada describe usa ChefSpec::SoloRunner con plataforma ubuntu y versión 22.04, y converge la receta descrita con los atributos que cada prueba necesita (por ejemplo los de wordpress en principal y mi_wordpress). De forma resumida se verifica lo siguiente:

- mi_apache::default instala el paquete apache2.
- mi_mysql::default instala el servidor mysql-server.
- mi_php::default instala el paquete php.
- mi_wordpress::default genera el archivo wp-config.php a partir de la plantilla definida en el cookbook.
- principal::default incluye la receta mi_apache::default (comprobación de orquestación).

- Además, existe el cookbook propio `mi_propia` con su receta default y su spec asociado, usado como ejemplo adicional de ChefSpec sobre un cookbook reducido.

Para ejecutar todas las pruebas unitarias desde la raíz del proyecto de debe ejecutar lo siguiente:

- `bundle install` (solo la primera vez o cuando cambie el Gemfile)
- `bundle exec rspec`

Al finalizar, RSpec muestra el número de ejemplos ejecutados y si alguno ha fallado.

```
PS C:\wamp64\www\unir-devops-chef-wordpress> bundle exec rspec
ffi-libarchive could not be loaded, libarchive is probably not installed on system, archive_file will not be available

mi_apache::default
  instala el paquete apache2

mi_mysql::default
  instala el servidor MySQL (mysql-server)

mi_php::default
  instala el paquete php

mi_propia::default
  ejemplo de un test que pasa
  fallo deliberado (1 no es 0) (FAILED - 1)

mi_wordpress::default
  genera wp-config.php desde plantilla

principal::default
  incluye la receta mi_apache::default

Failures:

  1) mi_propia::default fallo deliberado (1 no es 0)
     Failure/Error: expect(1).to eq(0)

     expected: 0
     got: 1

     (compared using ==)
     # ./spec/unit/recipes/mi_propia_default_spec.rb:13:in 'block (2 levels) in <top (required)>'

Finished in 0.34557 seconds (files took 1.9 seconds to load)
7 examples, 1 failure

Failed examples:

rspec ./spec/unit/recipes/mi_propia_default_spec.rb:12 # mi_propia::default fallo deliberado (1 no es 0)
```

Pruebas de integración

Para ser capaz de crear y completar las pruebas de integración es necesario realizar una serie de pasos:

Primero es necesario crear dos ficheros de configuración en la raíz del proyecto, el *Policyfile.rb* y el *kitchen.yml*:

- El *Policyfile.rb* contiene las dependencias y variables para hacer un despliegue en un ambiente de producción.

```
1  name 'chef-wordpress'
2  default_source :supermarket
3
4  # Configurar la ruta en donde se encuentran los Cookbooks
5  default_source :chef_repo, './chef/cookbooks'
6
7  # Asignar los atributos por defecto para el Cookbook de WordPress
8  default['wordpress']['url'] = 'http://localhost'
9  default['wordpress']['document_root'] = '/var/www/wordpress'
10 default['wordpress']['db_name'] = 'wordpress_db'
11 default['wordpress']['db_user'] = 'wp_user'
12 default['wordpress']['db_password'] = 'wp_password'
13 default['wordpress']['db_host'] = 'localhost'
14 default['wordpress']['title'] = 'TechOps Solutions, S.A'
15 default['wordpress']['admin_user'] = 'admin'
16 default['wordpress']['admin_password'] = 'admin123'
17 default['wordpress']['admin_email'] = 'admin@example.com'
18
19 # Ejecutar el Cookbook principal
20 run_list 'principal::default'
```

- El *kitchen.yml* configura las pruebas locales, indica las plataformas en las que se van a hacer pruebas y define las suites de pruebas.

```

1  ---
2  driver:
3    name: vagrant
4
5  provisioner:
6    name: chef_infra
7    cookbooks_path:
8      - chef/cookbooks
9    attributes:
10     wordpress:
11       url: "http://localhost"
12       document_root: "/var/www/wordpress"
13       db_name: "wordpress_db"
14       db_user: "wp_user"
15       db_password: "wp_password"
16       db_host: "localhost"
17       title: "TechOps Solutions, S.A"
18       admin_user: "admin"
19       admin_password: "admin123"
20       admin_email: "admin@example.com"
21
22  platforms:
23    - name: ubuntu-22.04
24
25  verifier:
26    name: inspec
27
28  suites:
29    - name: wordpress
30      run_list:
31        - recipe[principal::default]
32      verifier:
33        inspec_tests:
34          - "test/integration/wordpress"

```

Luego se crea la jerarquía de carpetas para las pruebas, esta es *test/integration/wordpress*.

Finalmente se crea el archivo con las pruebas de integración, *wordpress_spec.rb*. Las pruebas realizadas verifican lo siguiente:

- Que el servicio de Apache esté en funcionamiento.
- Que la configuración de la página web de Apache para WordPress exista.
- Que el servicio de MySQL esté en funcionamiento.
- Que las credenciales, y la base de datos, de WordPress se hayan creado correctamente.
- Que la página creada con WordPress este disponible y respondiendo solicitudes.

```

1  # Se verifica que el servicio de Apache este instalado, habilitado y en ejecucion
2  describe service('apache2') do
3    it { should be_installed }
4    it { should be_enabled }
5    it { should be_running }
6  end
7
8  # Se verifica que el archivo de configuracion de Apache para WordPress exista
9  describe file('/etc/apache2/sites-enabled/wordpress.conf') do
10   it { should exist }
11 end
12
13 # Se verifica que el servicio de MySQL este instalado, habilitado y en ejecucion
14 describe service('mysql') do
15   it { should be_installed }
16   it { should be_enabled }
17   it { should be_running }
18 end
19
20 # Se verifica que la base de datos de WordPress y sus credenciales existan en MySQL
21 describe mysql_session('wp_user','wp_password').query('SHOW DATABASES;') do
22   its('stdout') { should match /wordpress_db/ }
23 end
24
25 # Se verifica que la pagina de inicio de WordPress este disponible y respondiendo correctamente
26 describe http('http://localhost:80') do
27   its('status') { should cmp 200 }
28 end

```

Una vez toda la configuración es completada se pueden ejecutar las pruebas ejecutando el comando *kitchen test*.

```

Service apache2
  [PASS] is expected to be installed
  [PASS] is expected to be enabled
  [PASS] is expected to be running
File /etc/apache2/sites-enabled/wordpress.conf
  [PASS] is expected to exist
Service mysql
  [PASS] is expected to be installed
  [PASS] is expected to be enabled
  [PASS] is expected to be running
MySQL query: SHOW DATABASES;
  [PASS] stdout is expected to match /wordpress_db/
HTTP GET on http://localhost:80
  [PASS] status is expected to cmp == 200

Test Summary: 9 successful, 0 failures, 0 skipped
  Finished verifying <wordpress-ubuntu-2204> (0m7.07s).
-----> Destroying <wordpress-ubuntu-2204>...
  ==> default: Forcing shutdown of VM...
  ==> default: Destroying VM and associated drives...
Vagrant instance <wordpress-ubuntu-2204> destroyed.
  Finished destroying <wordpress-ubuntu-2204> (0m7.82s).
  Finished testing <wordpress-ubuntu-2204> (4m31.34s).
-----> Test Kitchen is finished. (4m36.38s)

```